

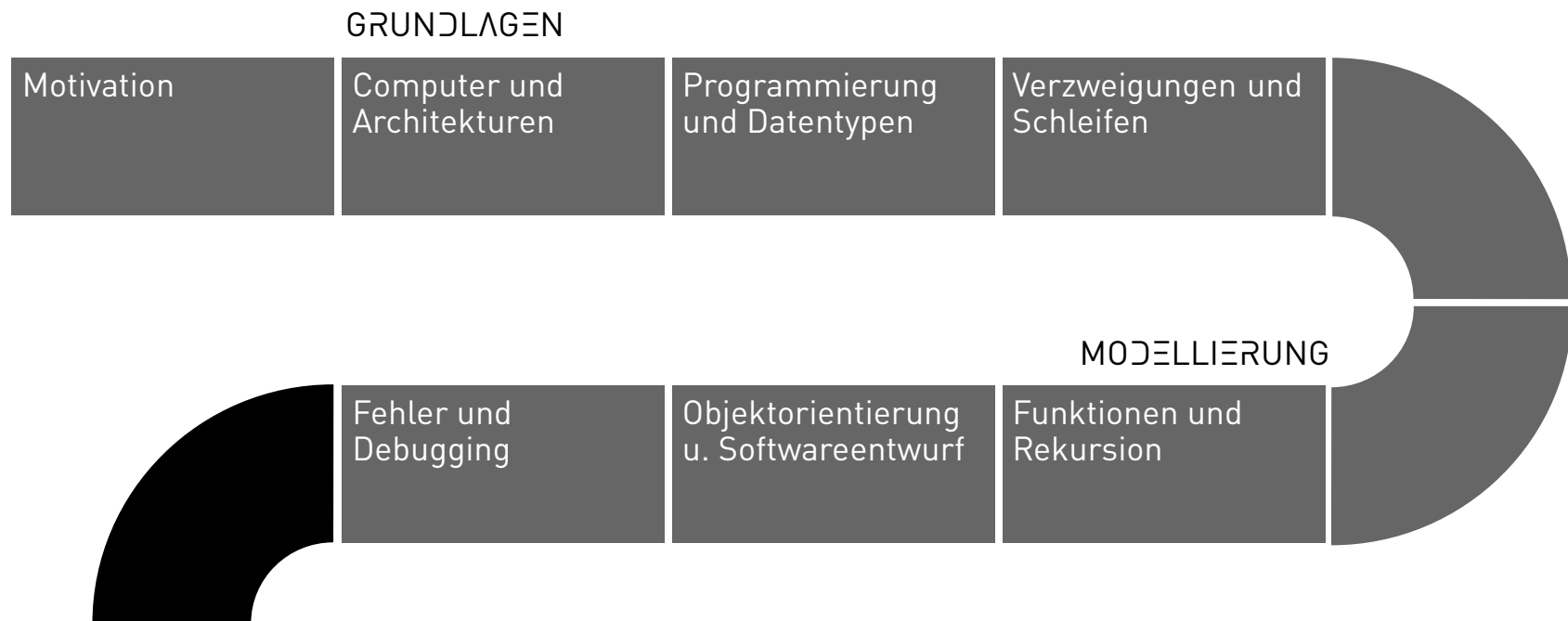
Programming and databases

Joern Ploennigs

Unit-Tests

MIDJOURNEY: WAVES TESTING A BOAT, REF. HOKUSAI

PROCEDURE



STRATEGIES FOR DEALING WITH PROGRAMMING ERRORS

5 main strategies:

1. Error prevention
2. Error detection
3. Bug fixing
4. Error handling
5. Error exclusion

Each strategy has its own methods and tools

1. ERROR PREVENTION

- Core principle: Start with knowledge of where errors can occur
- Strategies for safe code:
 - *Code Reviews* → Each line of code is reviewed by a different programmer
 - *Test-Driven Development* → You write the test before the actual code
 - *Full Test Coverage* → Every line of code should have at least one test
- Note: Prevention can require a lot of upfront work

2. ERROR DETECTION

Different types of errors require different tools

Syntax errors:

- Automatically detected in the IDE

Static errors:

- Via *lint tools* (static code analysis)
- Rule-based approaches
- Modern IDEs internally use lint tools

Dynamic errors:

- Checked by *automatically executed unit tests*
- After every code change
- Systematically test functions/methods

2. ERROR DETECTION – UNIT TESTS



Definition: Unit-Test

Unit tests are additional code written solely to test a module (function, class, module).

- Key properties:
 - Often the test code is more extensive than the code being tested
 - *Goal:* Verify that the module correctly performs the desired function
 - *White-box test:* Call the module with specific inputs
 - Check expected outputs or exceptions
 - Should be executed automatically after every code change

2. ERROR DETECTION – UNIT TEST TYPES

Functional Tests

Correct implementation of the function

Example: Division module

```
numerator = 10
denominator = 2
result =
division(numerator,
denominator)
assert result == 5 #
Expected: 5
```

Boundary Value Tests

Correct handling of boundary values

```
numerator = 10
denominator = 0
result =
division(numerator,
denominator)
assert result == None #
Expected: None
```

Data Type Tests

Correct handling of unexpected inputs

```
numerator =
'not_a_number'
denominator = 0
try:
    result =
division(numerator,
denominator)
    assert False # We
let the test fail
except TypeError as e:
    # Expected:
    TypeError
```

3. DEBUGGING - FUNDAMENTALS



Definition: Debugging

Debugging is the systematic search for and fixing of errors.

- Methods:
 - Logging messages to trace the program flow
 - Debugger for detailed analysis
- Debugger features:
 - Step-by-step execution of code lines
 - Monitoring variable values
 - Dynamically inserting code

3. DEBUGGING IN PYTHON

- Systematic debugging is a core skill for every programmer
- Python debugging tools
 - Standard debugger: pdb (Python module)
 - IDE-specific debugger (often more user-friendly)
- More details on this in the exercise!

4. ERROR HANDLING - READY FOR ANYTHING

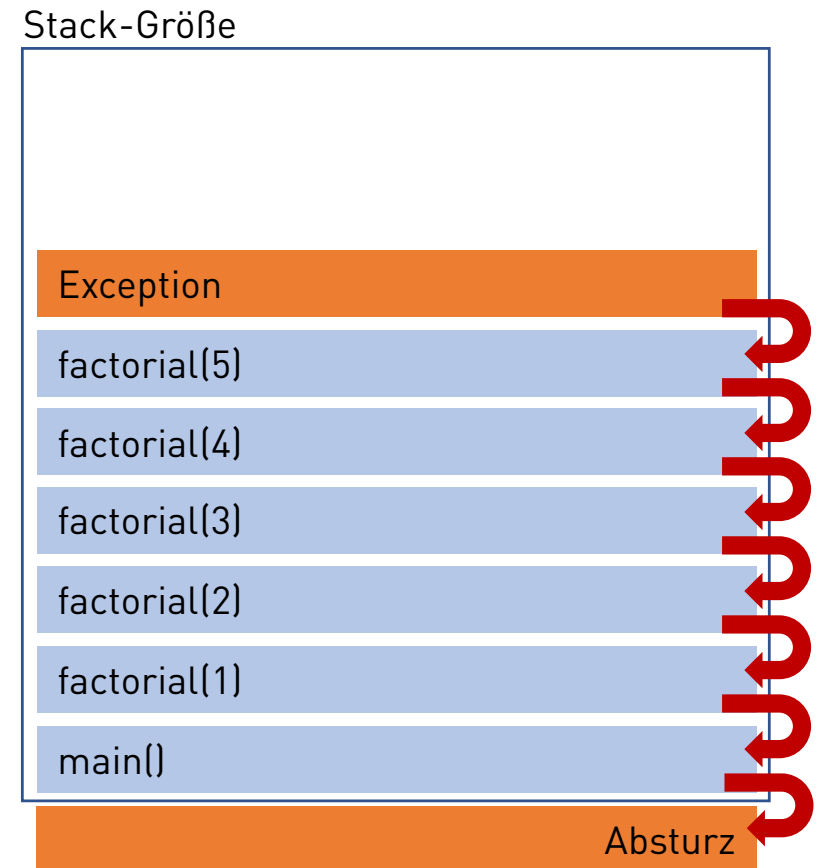
- Reality: Despite all efforts, runtime errors will occur in real-world programs
- Goal:
 - Catch and handle errors
 - The program continues to run or terminates in a controlled manner
- Mechanism:
 - Runtime error → *Exception* is raised
 - Python tools: `try`, `except`, `raise`

4. ERROR HANDLING - IN PYTHON

- **try:** Contains a block of code that may raise an exception
- **except:** If a specific error (Exception) occurs in the try block, it is caught and the corresponding except block is executed
- **raise:** Is used inside an except block to re-raise an exception
- **finally:** Is executed after the end of the try block, regardless of whether an error occurred
- **else:** Optional, defined after all the except blocks; it runs if no exception occurs in the try block

SEMANTIC ERRORS - EXCEPTIONS

- *Exception-Mechanismus*:
 - Interrupt normal program flow
 - Communicate errors with an error message
 - Prevent uncontrolled crashes
 - Are propagated up the *call stack*
 - Until they are caught or the program crashes
- Goal: keep the program in a running state



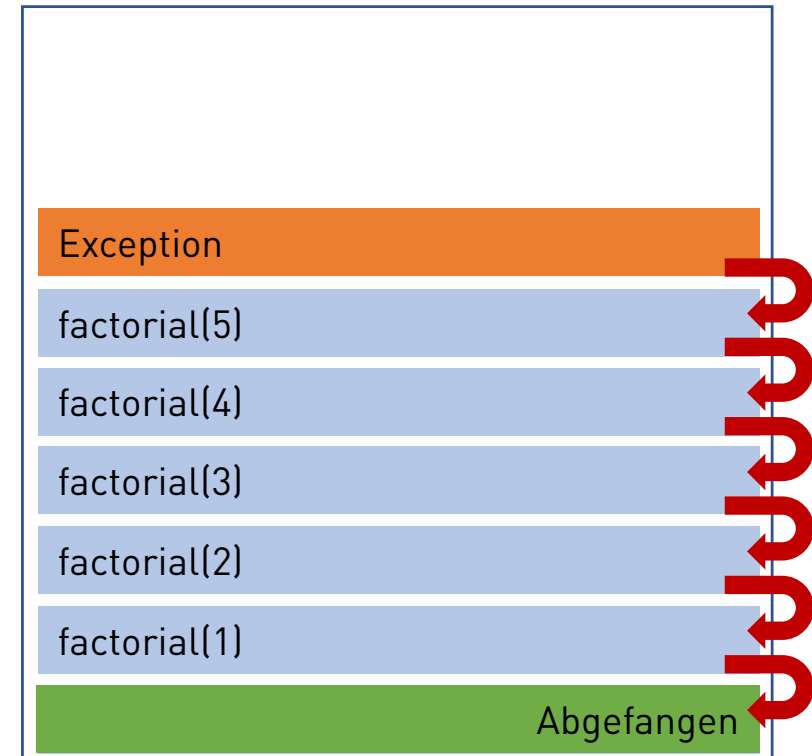
SEMANTIC ERRORS – CATCHING EXCEPTIONS

Expected exceptions should always be caught

Python `try-except` block:

```
try:
    # Block with the expected
    exception
    ergebnis = zaehler / nenner
except:
    # Error handling
    print("Division by 0")
    ergebnis = None
```

Stack-Größe



SEMANTIC ERRORS - EXCEPTION LEVELS

Exceptions are used at multiple levels:

- Operating system - Predefined system exceptions
- Programming language - Built-in language exceptions
- Custom code - Self-defined exceptions

Use cases (where errors are likely to occur):

- Communication with external sources
- Reading files
- User input
- Network connections

4. ERROR HANDLING - EXAMPLE: DIVISION BY ZERO

```
zaehler = 12
nenner = 0

try:
    ergebnis = zaehler / nenner
except:
    print("Exception")
```

Question: Which exceptions can be raised here?

4. ERROR HANDLING - EXAMPLE: DIVISION BY 0

```
zaehler = 12
nenner = 0

try:
    ergebnis = zaehler / nenner
except:
    print("Exception")
```

Question: Which exceptions can be raised here?

Possible exceptions:

- `ZeroDivisionError` - division by zero
- `TypeError` - wrong data type
- `NameError` - variable not defined

4. ERROR HANDLING - PROPAGATING EXCEPTIONS

- An important concept in more complex software
- We can handle errors locally, but we still want to inform the higher levels of the program about them.
- The `raise` statement propagates an exception upward from an `except` block.

4. ERROR HANDLING - EXAMPLE: PROPAGATING AN EXCEPTION

```
def count_up_and_down(x):  
    if x <= 0:  
        raise ValueError("No negative values allowed")  
  
    # Verknüpft vorwärts- und rückwärts-Liste  
    return list(range(x)) + list(reversed(list(range(x-1))))  
  
try:  
    print(count_up_and_down(-6))  
except ValueError:  
    print("That value was invalid.")
```

5. ERROR ELIMINATION — PROVING PROGRAM CORRECTNESS MATHEMATICALLY

- Some program code can be validated through *mathematical proof*
- *Hoare calculus*: Formalize the program line by line with mathematical logic
- From the beginning to the end of the entire program
- *Problem*: For most applications, too time-consuming and complex
- *Use*: Only for critical systems (medical technology, aviation, etc.)

Questions?

